

UNIVERSIDAD NACIONAL DE ASUNCIÓN

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA MECATRÓNICA

Sistema de adquisición y monitoreo de nivel de agua

Informe técnico de avance del proyecto

Profesores:

Prof. Ing. Federico Gaona, MSc.

Prof. Ing. Adolfo Monje

Alumnos:

Héctor Dejesús Velázquez Ojeda 5.011.166

Mathias Ramon Aguilar DelValle 4.970.804

Mauricio Ivan Tullo Estigarribia 3.782.101

San Lorenzo, Paraguay

2026

Índice

1. Introducción	3
2. Objetivos del proyecto	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Descripción general del sistema	3
3.1. Componentes principales del sistema	4
3.2. Sistema de alimentación	4
3.3. Asignación de pines del sistema	5
4. Procedimiento de montaje y horneado de las PCBs	6
4.1. Revisión inicial de la PCB	6
4.2. Preparación de los componentes	6
4.3. Aplicación de pasta de soldar	6
4.4. Colocación de componentes en la PCB	7
4.5. Proceso de horneado	8
4.6. Inspección posterior al horneado	9
5. Problemas encontrados y soluciones	10
5.1. Evidencia del reemplazo del RTC DS3231+TRL por DS3231M	11
6. Avances de la programación	12
6.1. Entorno de desarrollo utilizado	12
6.2. Estructura de carpetas del proyecto	13
6.3. Librerías y componentes utilizados	13
6.3.1. Librerías generales de C	13
6.3.2. Librerías principales de ESP-IDF	14
6.3.3. Librerías FreeRTOS	15
6.3.4. Librerías de drivers ESP-IDF	16
6.3.5. Librerías para SD Card	17
6.3.6. Componentes propios del proyecto	17
6.4. Módulos desarrollados	19
7. Flujo conceptual del algoritmo final	20
7.1. Descripción del flujo	20
8. Resultados de pruebas aisladas de módulos del sistema	24
8.1. Prueba aislada del sensor de nivel RS485	24
8.2. Prueba aislada del DS3231M	24
8.3. Prueba aislada de la tarjeta SD	25
8.4. Prueba aislada del módem DTU	26
8.5. Prueba integrada del sistema completo	27
9. Estado actual del proyecto	28
10. Repositorio GitHub	29

11.Conclusión	29
12.Trabajos futuros	30
13.Referencias	31
14.Anexos	32
14.1. Capturas adicionales del sistema	32

1. Introducción

En este informe se presenta el avance del proyecto denominado *Sistema de adquisición y monitoreo de nivel de agua*. El documento describe el procedimiento realizado para el montaje y horneado de las placas PCB, los problemas encontrados durante el proceso y las soluciones aplicadas.

Además, se presentan los avances realizados en la programación del sistema embebido, desarrollado en Visual Studio Code utilizando PlatformIO y el framework ESP-IDF. Finalmente, se incluyen los resultados obtenidos en las pruebas aisladas de sensores.

2. Objetivos del proyecto

2.1. Objetivo general

Desarrollar un sistema electrónico capaz de adquirir, procesar y monitorear datos de nivel de agua mediante sensores conectados a un microcontrolador, utilizando una PCB diseñada para integrar los distintos módulos del sistema.

2.2. Objetivos específicos

- Realizar el montaje de los componentes electrónicos sobre la PCB.
- Aplicar el proceso de horneado o soldadura por reflow para componentes SMD.
- Verificar el correcto funcionamiento eléctrico de la placa.
- Desarrollar el firmware del sistema utilizando PlatformIO y ESP-IDF.
- Probar de forma aislada los sensores del sistema.
- Documentar los problemas encontrados y las soluciones aplicadas.
- Mantener actualizado el repositorio GitHub con los códigos desarrollados.

3. Descripción general del sistema

El sistema está compuesto por una placa electrónica principal, un microcontrolador ESP32 [1], sensores para la adquisición de datos y módulos auxiliares para comunicación, almacenamiento o alimentación, según los requerimientos del proyecto.

El firmware se desarrolla en lenguaje C mediante el entorno Visual Studio Code con PlatformIO [3], utilizando el framework ESP-IDF [2]. Esta elección permite trabajar con una estructura modular, configurar periféricos del ESP32 y utilizar herramientas propias del ecosistema Espressif.

Este proyecto se relaciona con trabajos previos orientados al monitoreo y predicción de nivel en el arroyo Mburicao, donde se analizaron modelos de aprendizaje automático aplicados a eventos de nivel de agua [4].

3.1. Componentes principales del sistema

Componente	Función dentro del sistema
ESP32-S3-WROOM-1	Microcontrolador principal encargado de controlar el sistema.
PCB diseñada	Placa electrónica donde se integran los componentes del sistema.
Sensor de nivel de agua	Permite medir el nivel de agua para el monitoreo.
Módulo de comunicación	Permite enviar datos hacia un sistema externo o servidor.
Fuente de alimentación UPS	Proporciona la energía necesaria para el funcionamiento del sistema.

Cuadro 1: Componentes principales del sistema.

3.2. Sistema de alimentación

El sistema de alimentación del proyecto se basa en una mini UPS inteligente **Marsriva Serie KP2** [11], la cual cuenta con un tiempo de transferencia de 0 ms que asegura la continuidad operativa absoluta del sistema ante cualquier corte del suministro eléctrico principal. A partir de las salidas independientes de esta fuente principal, se distribuyen los diferentes niveles de tensión requeridos por los módulos del sistema:

- **12 V:** Utilizados para la alimentación del módem DTU GPRS y el sensor ultrasónico de nivel.
- **5 V:** Utilizados para la alimentación del microcontrolador ESP32-S3, ya sea mediante conector jack o puerto USB-C.
- **3.3 V:** Regulados internamente por la placa para alimentar los periféricos de baja potencia.

La placa incluye indicadores LED de estado de alimentación:

- **LED rojo:** Indica la presencia de tensión de 5 V (VCC) en el sistema.
- **LED verde:** Indica la correcta regulación y disponibilidad de 3.3 V para el ESP32 y sus periféricos.
- **LED azul:** Indica la actividad operativa del sistema, destellando durante la ejecución de la lectura del sensor ultrasónico o durante el proceso de transmisión de datos a través del módem GPRS.

Estos indicadores permiten una verificación rápida del estado eléctrico del sistema durante la instalación y mantenimiento. La Figura 1 muestra la mini UPS utilizada en el proyecto como fuente principal de alimentación.

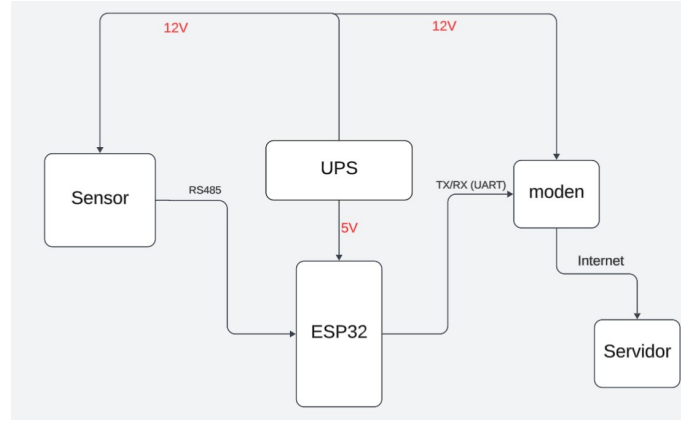


Figura 1: Mini UPS utilizada como fuente principal de alimentación.

3.3. Asignación de pines del sistema

En el Cuadro 2 se presenta la asignación de pines utilizada para la conexión de los módulos principales del sistema con el ESP32-S3. Esta tabla permite identificar las señales correspondientes al sensor de nivel RS485 [6], módem DTU [5], tarjeta microSD [8], reloj DS3231M [7] y LED indicador.

Cuadro 2: Asignación de pines utilizados en el sistema.

Módulo	Señal	GPIO ESP32-S3	Interfaz	Función
Sensor de nivel RS485	TX / DI1	GPIO1	UART1 TX	Envío de comando Modbus al sensor de nivel.
Sensor de nivel RS485	RX / RO1	GPIO2	UART1 RX	Recepción de la respuesta enviada por el sensor.
Sensor de nivel RS485	DE/RE1	GPIO4	Control RS485	Control de transmisión y recepción del bus RS485.
Módem DTU	TX / DI2	GPIO5	UART2 TX	Envío de la trama de datos hacia el módem DTU.
Módem DTU	RX / RO2	GPIO6	UART2 RX	Recepción de la respuesta del módem o del servidor.
Módem DTU	DE/RE2 / RTS	GPIO7	Control RS485	Control de dirección de comunicación RS485 del módem.
DS3231M	SDA	GPIO8	I2C0 SDA	Línea de datos para la comunicación con el RTC.
DS3231M	SCL	GPIO9	I2C0 SCL	Línea de reloj para la comunicación con el RTC.
DS3231M	SQW/INT	GPIO16	Entrada digital	Señal de interrupción o pulso del RTC, reservada para uso futuro.
Tarjeta microSD	CS	GPIO10	SPI CS	Señal de selección de la tarjeta microSD.
Tarjeta microSD	MOSI	GPIO11	SPI MOSI	Envío de datos desde el ESP32-S3 hacia la microSD.
Tarjeta microSD	CLK	GPIO12	SPI CLK	Señal de reloj del bus SPI.
Tarjeta microSD	MISO	GPIO13	SPI MISO	Recepción de datos desde la microSD hacia el ESP32-S3.
Tarjeta microSD	DET	GPIO15	Entrada digital	Detección de presencia de tarjeta microSD.
LED indicador	LED Blink	GPIO21	Salida digital	Indicador visual de lectura válida del sensor de nivel.

4. Procedimiento de montaje y horneado de las PCBs

4.1. Revisión inicial de la PCB

Antes de iniciar el montaje de los componentes, se realizó una inspección visual de la PCB con el objetivo de verificar el estado general de la placa. Se revisaron las pistas, pads, perforaciones, serigrafía y posibles defectos de fabricación.

También se verificó la correspondencia entre el diseño esquemático, el diseño PCB y la ubicación física de los componentes.

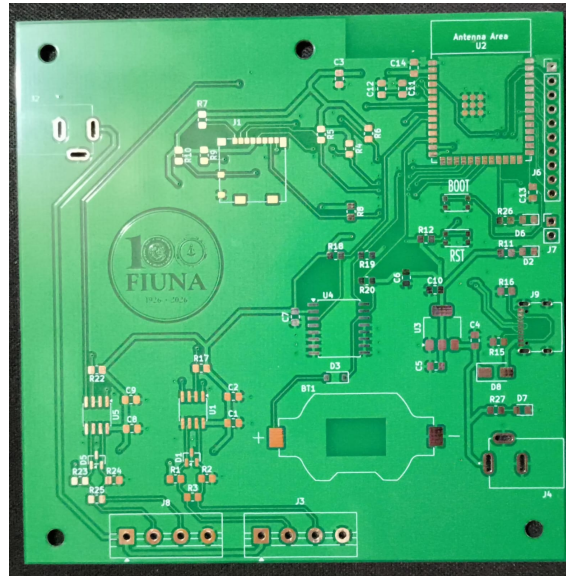


Figura 2: Revisión inicial de la PCB antes del montaje de los componentes.

4.2. Preparación de los componentes

Los componentes electrónicos fueron organizados de acuerdo con su tipo, encapsulado y ubicación en la placa. Se separaron los componentes SMD de los componentes de montaje tradicional, con el fin de facilitar el proceso de ensamblaje.

4.3. Aplicación de pasta de soldar

Para la aplicación de la pasta de soldar Sn63Pb37 tipo 4 se utilizó un *stencil*, el cual permitió depositar la pasta únicamente en los lugares donde era necesaria, específicamente sobre los pads de la PCB correspondientes a los componentes SMD.

Antes de aplicar la pasta, la PCB fue fijada sobre la mesa de trabajo para evitar movimientos durante el proceso. Luego, el *stencil* fue ubicado por encima de la placa, alineando cuidadosamente sus aberturas con los pads de soldadura. Una vez alineado, se procedió a distribuir la pasta de soldar utilizando una espátula, aplicando presión de forma uniforme con un ángulo aproximado de 45°.

Después de retirar el *stencil*, se observó la PCB con ayuda de un microscopio para verificar cómo quedó depositada la pasta sobre los pads. En esta inspección se revisó que la pasta estuviera correctamente ubicada en las zonas de conexión, evitando excesos, falta de material o posibles uniones entre pads cercanos.

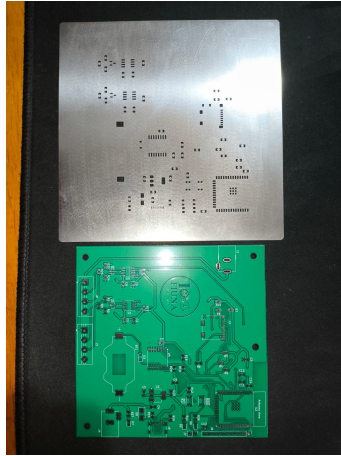


Figura 3: Stencil utilizado para la aplicación de pasta de soldar sobre los pads de la PCB.

4.4. Colocación de componentes en la PCB

Luego de aplicar la pasta de soldar, se procedió a colocar los componentes sobre la PCB utilizando pinzas de precisión y tomando como referencia la serigrafía de la placa. Durante esta etapa se verificó la correcta posición y orientación de los componentes, especialmente en aquellos que presentan polaridad o una orientación definida, como diodos, circuitos integrados, conectores y módulos.

El primer componente colocado fue el módulo ESP32-S3-WROOM-1-N8R8 [1], debido a que corresponde al componente principal de control del sistema y requiere una correcta alineación con sus respectivos pads de soldadura. Al tratarse de un módulo con múltiples terminales, se verificó cuidadosamente su posición antes del proceso de horneado para evitar desalineaciones o problemas de contacto eléctrico.

Posteriormente, se ubicaron los componentes de menor tamaño, comenzando por los capacitores y luego los resistores. A continuación, se colocaron los diodos y, finalmente, los componentes de mayor tamaño o con mayor complejidad mecánica, como el conector para la tarjeta SD [8] y el conector USB [9].

El portapilas no fue colocado durante esta etapa, ya que se decidió evitar su exposición al proceso de horneado. Esta decisión se tomó para reducir el riesgo de daño térmico en dicho componente, por lo que su montaje se dejó para una etapa posterior mediante soldadura manual.

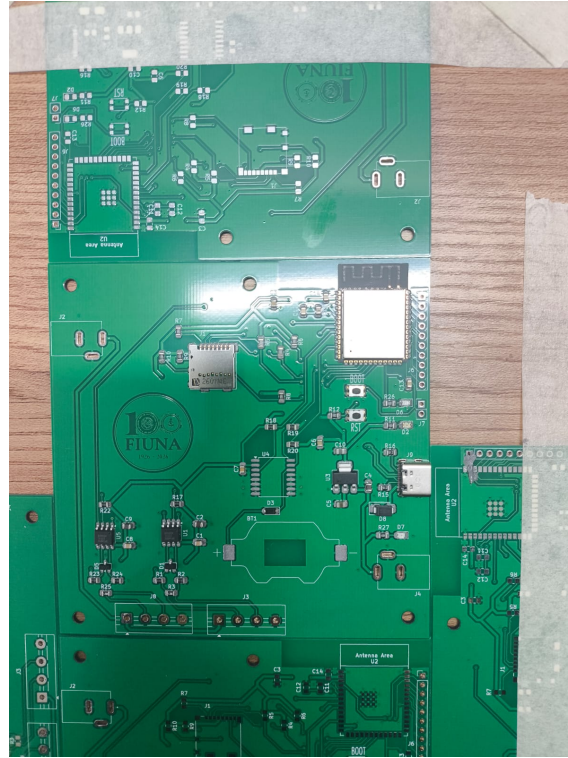


Figura 4: Colocación de los componentes sobre la PCB utilizando pinzas de precisión.

4.5. Proceso de horneado

Una vez ubicados los componentes, la placa fue introducida en el cajón de inspección del horno de reflujo infrarrojo *T962C* [10]. Este equipo opera mediante un control por microprocesador que combina la emisión de radiación infrarroja potente con la circulación de un flujo de aire caliente, garantizando una distribución precisa y uniforme de la temperatura en toda la superficie de la PCB.

El proceso se ejecutó de manera completamente automática. Para este proyecto en particular, se configuró el equipo en el ciclo térmico predefinido **Wave 2**. Según el manual del fabricante, esta curva está diseñada de forma específica para trabajar con la aleación de la pasta de soldar utilizada en el montaje (**SN63/PB37 Tipo 4**). El ciclo seleccionado gestiona de forma autónoma cuatro etapas críticas:

- **Preheat (Precalentamiento):** Elevación inicial y progresiva de la temperatura desde el estado ambiente.
- **Soak (Estabilización / Activación):** Homogeneización térmica de toda la placa y activación química de los componentes del flux de la pasta.
- **Reflow (Reflujo / Fusión):** Fase de rampa máxima donde se alcanza la temperatura pico logrando la fundición total de la aleación Sn63/Pb37.
- **Cool down (Enfriamiento):** Descenso térmico controlado mediante la activación automática del ventilador de extracción integrado en el equipo.

Durante todo el procedimiento, la formación de la curva y las lecturas térmicas internas se monitorearon en tiempo real a través de la pantalla LCD del panel de

control. Una vez completado el ciclo automatizado, el sistema detuvo el calentamiento, activó la ventilación de escape para enfriar la placa y emitió una alerta sonora indicando la conclusión segura del proceso.

Etapas del Ciclo T962C	Perfil Térmico (Wave 2)	Acción del equipo
Preheat (Precalentamiento)	Rampa de calentamiento inicial	Infrarrojo y aire caliente activos
Soak (Estabilización)	Meseta de estabilización térmica	Activación del flux
Reflow (Reflujo o Fusión)	Pico máximo para soldadura Sn/Pb	Fundición total automatizada
Cool down (Enfriamiento)	Descenso térmico progresivo	Ventilador extractor activo y alarma

Cuadro 3: Etapas secuenciales del ciclo de soldado automatizado (Wave 2) del horno T962C para la pasta SN63/PB37.

4.6. Inspección posterior al horneado

Después del horneado, se realizó una inspección visual de las soldaduras. Se verificó que los componentes estuvieran correctamente alineados, que no existieran puentes de estaño y que las uniones fueran uniformes.

En los casos necesarios, se realizaron correcciones manuales utilizando cautín, flux y malla desoldadora.

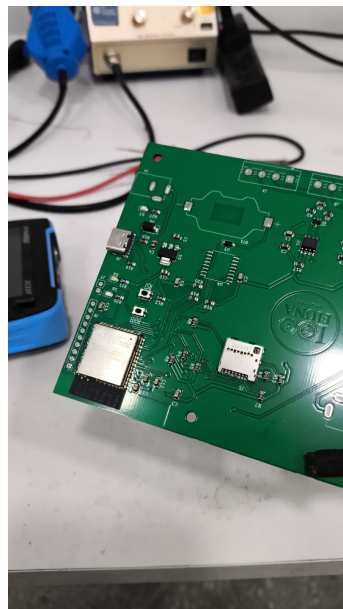


Figura 5: Inspección posterior al horneado de la PCB para verificar la calidad de las soldaduras y la correcta alineación de los componentes.

5. Problemas encontrados y soluciones

Durante el proceso de montaje, horneado y pruebas iniciales se encontraron distintos problemas.

Problema encontrado	Causa probable	Solución aplicada o pendiente
Exceso de soldadura en algunos pads	Aplicación excesiva de pasta de soldar	Se retiró el exceso utilizando cautín, flux y malla desoldadora.
Puentes de soldadura entre terminales cercanos	Exceso de pasta de soldar o mala distribución durante el montaje	Se corrigió con malla desoldadora y se verificó con multímetro.
Falta del circuito integrado RTC DS3231+TRL	El componente DS3231+TRL fue solicitado para el montaje de la PCB, pero no llegó. Esto impidió montar directamente el componente nuevo sobre la placa.	Se utilizó como solución un módulo RTC comercial basado en el DS3231M. Antes de realizar el reemplazo, se compararon los datasheets del componente requerido y del componente presente en el módulo comercial, verificando la coincidencia de pines y características principales. Luego, se procedió a desoldar cuidadosamente el DS3231M del módulo comercial y posteriormente soldarlo en la PCB del proyecto.
Pines puenteados en el conector USB	Durante el proceso de soldadura se generaron puentes de estaño entre algunos pines del conector USB, posiblemente debido al exceso de pasta de soldar o a la cercanía entre terminales.	Se corrigió el problema utilizando cautín y malla desoldadora de cobre para retirar el exceso de estaño. Luego se realizó una inspección visual y verificación de continuidad para confirmar que los pines ya no estuvieran en cortocircuito.
Pines puenteados en el conector de la tarjeta SD	Al igual que en el conector USB, se observaron puentes de soldadura entre pines del conector SD debido al exceso de estaño durante el proceso de reflow o corrección manual.	Se aplicó el mismo procedimiento de corrección, utilizando cautín y malla desoldadora de cobre para eliminar el exceso de estaño. Posteriormente, se verificó que no existieran cortocircuitos entre pines adyacentes.

Problema encontrado	Causa probable	Solución aplicada o pendiente
Problema de configuración del módem GPRS DTU	El módem GPRS DTU cuenta con un software propio para su configuración mediante comandos AT. Sin embargo, durante las pruebas iniciales no se logró que el dispositivo fuera reconocido correctamente mediante el cable micro USB. Se probaron distintos cables y conexiones, pero el problema persistió, impidiendo realizar la configuración desde el software del fabricante.	Como solución, se desarrolló un script desde PlatformIO para enviar directamente al módem los comandos AT necesarios para su configuración. Mediante este procedimiento se cargaron los parámetros requeridos para la comunicación y el envío de datos al servidor, utilizando la trama correcta del sistema. Esta solución permitió configurar el módem y habilitar el envío exitoso de datos al servidor.

Cuadro 4: Problemas encontrados y soluciones aplicadas o pendientes.

5.1. Evidencia del reemplazo del RTC DS3231+TRL por DS3231M

Ante el retraso en la entrega del componente DS3231M+TRL, se optó por extraer el circuito integrado DS3231M de un módulo RTC comercial disponible. Para validar esta solución, se realizó la comparación de las hojas de datos del componente, verificándose que el dispositivo utilizado mantiene la misma funcionalidad y el mismo encapsulado que el componente previsto originalmente. La diferencia principal corresponde al sufijo +TRL, asociado al método de empaque de fábrica para distribución en cinta y carrete.

Posteriormente, el circuito integrado fue desoldado cuidadosamente del módulo comercial y transferido a la PCB desarrollada para el proyecto. Esta solución permitió continuar con el montaje de la placa sin modificar el diseño original del circuito, manteniendo la compatibilidad eléctrica y funcional del sistema RTC [7].

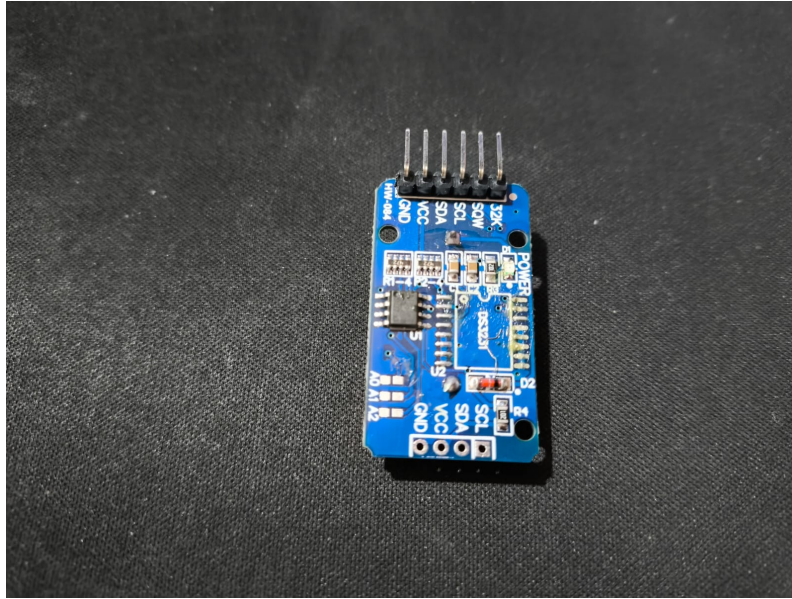


Figura 6: Módulo RTC comercial luego de retirar el circuito integrado DS3231. Este componente fue desoldado para ser reutilizado en la PCB del proyecto.

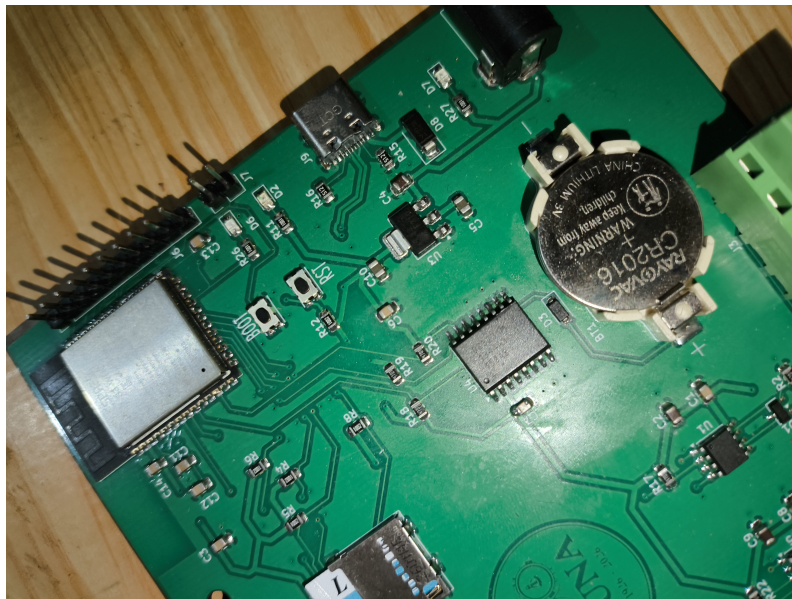


Figura 7: Circuito integrado DS3231 soldado en la PCB del proyecto como solución al problema del componente faltante.

6. Avances de la programación

6.1. Entorno de desarrollo utilizado

La programación del sistema se realiza en Visual Studio Code utilizando PlatformIO [3] como entorno de desarrollo y ESP-IDF [2] como framework principal. Esta configuración permite trabajar de forma ordenada con el ESP32 [1], utilizando herramientas de compilación, carga y monitoreo serial integradas.

6.2. Estructura de carpetas del proyecto

El proyecto fue organizado de forma modular, separando el programa principal de los componentes específicos del sistema. Cada componente cuenta con sus propios archivos fuente, cabecera y archivo `CMakeLists.txt`, lo que permite una mejor organización, mantenimiento y reutilización del código.

```
Firmware_Final/  
  components/  
    led/  
      CMakeLists.txt  
      led_blink.c  
      led_blink.h  
    modem_dtu/  
      CMakeLists.txt  
      modem_dtu.c  
      modem_dtu.h  
    reloj/  
      CMakeLists.txt  
      reloj.c  
      reloj.h  
    sdcard/  
      CMakeLists.txt  
      sdcard.c  
      sdcard.h  
    sensor_rs485/  
      CMakeLists.txt  
      sensor_rs485.c  
      sensor_rs485.h  
  include/  
  lib/  
  src/  
    CMakeLists.txt  
    main.c  
  test/  
  .gitignore  
  CMakeLists.txt  
  platformio.ini  
  sdkconfig.freenove_esp32_s3_wroom
```

6.3. Librerías y componentes utilizados

En esta sección se describen las principales librerías utilizadas en el firmware del proyecto desarrollado con ESP-IDF [2]. Para una mejor organización, se agrupan según su función dentro del sistema.

6.3.1. Librerías generales de C

Las librerías generales de C permiten utilizar funciones básicas del lenguaje, tales como entrada y salida estándar, manejo de tipos de datos, variables booleanas,

cadenas de caracteres y conversión de fecha y hora.

Librería	Dónde se usa	Función en el proyecto
stdio.h	main.c, sdcard.c, reloj.c, modem_dtu.c	Permite utilizar funciones como <code>printf()</code> , <code>fprintf()</code> , <code>snprintf()</code> , <code>fopen()</code> y <code>fclose()</code> , necesarias para impresión de mensajes, formateo de cadenas y lectura/escritura de archivos.
stdint.h	main.c, reloj.h, sdcard.h, modem_dtu.h	Define tipos de datos enteros de tamaño fijo, como <code>uint8_t</code> , <code>uint16_t</code> y <code>uint32_t</code> , utilizados para trabajar de forma segura con datos, tiempos, registros y variables del sistema.
stdbool.h	Varios archivos .h y .c	Permite utilizar el tipo de dato booleano <code>bool</code> , junto con los valores <code>true</code> y <code>false</code> , facilitando la representación de estados lógicos dentro del programa.
string.h	sdcard.c, modem_dtu.c	Permite utilizar funciones para el manejo de cadenas de caracteres, como <code>strlen()</code> y <code>strstr()</code> , empleadas para comparar, buscar o procesar respuestas y datos de texto.
time.h	reloj.c	Permite trabajar con estructuras y funciones de tiempo. En el proyecto se utiliza para convertir la fecha y hora obtenida desde el RTC a formato <i>timestamp</i> UNIX mediante la función <code>mktime()</code> .

Cuadro 5: Librerías generales de C utilizadas en el proyecto.

6.3.2. Librerías principales de ESP-IDF

Las librerías principales de ESP-IDF [2] permiten utilizar funciones propias del framework de Espressif, tales como el registro de mensajes por consola, el manejo de códigos de error y la generación de retardos precisos en microsegundos.

Librería	Dónde se usa	Función en el proyecto
esp_log.h	Casi todos los componentes	Permite mostrar mensajes de información, advertencia y error en el monitor serial mediante macros como <code>ESP_LOGI()</code> , <code>ESP_LOGW()</code> y <code>ESP_LOGE()</code> . Esta librería facilita la depuración del firmware durante las pruebas del sistema.

Librería	Dónde se usa	Función en el proyecto
esp_err.h	sdcard.h	Permite manejar códigos de error propios de ESP-IDF, como ESP_OK y ESP_FAIL. Se utiliza para indicar si una función fue ejecutada correctamente o si ocurrió una falla durante el proceso.
esp_rom_sys.h	sensor_rs485.c	Permite utilizar la función esp_rom_delay_us(), empleada para generar retardos cortos en microsegundos. En el proyecto se utiliza principalmente en rutinas donde se requieren tiempos breves y controlados, como en la comunicación con el sensor RS485.

Cuadro 6: Librerías principales de ESP-IDF utilizadas en el proyecto.

6.3.3. Librerías FreeRTOS

Las librerías de FreeRTOS permiten trabajar con tareas, retardos y mecanismos de sincronización dentro del firmware. En este proyecto se utilizan principalmente para organizar la ejecución del programa, temporizar operaciones y evitar conflictos cuando dos partes del sistema intentan usar un mismo recurso al mismo tiempo.

Librería	Dónde se usa	Función en el proyecto
freertos/ FreeRTOS.h	main.c sensor_rs485.c modem_dtu.c led_blink.c	Es la librería base de FreeRTOS. Permite utilizar las definiciones principales del sistema operativo en tiempo real, necesarias para el manejo de tareas, retardos y temporización del firmware.
freertos/task.h	main.c, sensor_rs485.c, modem_dtu.c, led_blink.c	Permite utilizar funciones relacionadas con tareas y retardos, como vTaskDelay(), xTaskCreate() y pdMS_TO_TICKS(). En el proyecto se utiliza para generar esperas controladas y ejecutar tareas en segundo plano.

Librería	Dónde se usa	Función en el proyecto
freertos/semphr.h	main.c	Permite utilizar mecanismos de sincronización como mutex mediante SemaphoreHandle_t, xSemaphoreCreateMutex(), xSemaphoreTake() y xSemaphoreGive(). En el proyecto se utiliza para evitar que el ciclo principal y la tarea de pendientes utilicen el módem DTU al mismo tiempo.

Cuadro 7: Librerías FreeRTOS utilizadas en el proyecto.

6.3.4. Librerías de drivers ESP-IDF

Las librerías de drivers de ESP-IDF permiten configurar y controlar directamente los periféricos internos del ESP32-S3, tales como pines GPIO, buses de comunicación I2C, UART y SPI. En este proyecto se utilizan para la comunicación con el RTC DS3231, el sensor RS485, el módem DTU, la tarjeta SD y señales digitales auxiliares.

Librería	Dónde se usa	Función en el proyecto
driver/gpio.h	sdcard.h, reloj.h, sensor_rs485.c, led_blink.h	Permite configurar y controlar los pines GPIO del ESP32-S3. Se utiliza para definir entradas, salidas, resistencias de <i>pull-up</i> y operaciones de lectura o escritura digital.
driver/i2c.h	reloj.h, reloj.c	Permite configurar el bus I2C utilizado para la comunicación entre el ESP32-S3 y el DS3231M. Mediante esta librería se realiza la lectura y escritura de registros internos del reloj.
driver/uart.h	sensor_rs485.c, modem_dtu.c, modem_dtu.h	Permite configurar los puertos UART del ESP32-S3. En el proyecto se utiliza para la comunicación serial con el sensor de nivel mediante RS485 y con el módem DTU.
driver/ spi_common.h	sdcard.c	Permite configurar aspectos comunes del bus SPI. En el proyecto se utiliza como parte de la inicialización del bus SPI empleado para la comunicación con la tarjeta SD.

Librería	Dónde se usa	Función en el proyecto
driver/ sdspi_host.h	sdcard.c	Permite utilizar la tarjeta SD en modo SPI mediante el controlador SDSPI de ESP-IDF. Esta librería facilita la comunicación entre el ESP32-S3 y la tarjeta de memoria.

Cuadro 8: Librerías de drivers ESP-IDF utilizadas en el proyecto.

6.3.5. Librerías para SD Card

Las librerías relacionadas con la tarjeta SD permiten montar la memoria externa como un sistema de archivos FAT y realizar operaciones de lectura y escritura. En este proyecto se utilizan para almacenar localmente las mediciones de nivel de agua en caso de ser necesario.

Librería	Dónde se usa	Función en el proyecto
esp_vfs_fat.h	sdcard.c	Permite montar la tarjeta SD como un sistema de archivos FAT dentro del ESP32-S3. En el proyecto se utiliza para acceder a la tarjeta mediante la ruta <code>/sdcard</code> , permitiendo crear, abrir, escribir y cerrar archivos.
sdmmc_cmd.h	sdcard.c	Permite inicializar, montar e imprimir información de la tarjeta SD. En el proyecto se utiliza junto con el modo SDSPI para reconocer la tarjeta, obtener sus características y habilitar su uso como medio de almacenamiento.

Cuadro 9: Librerías utilizadas para el manejo de la tarjeta SD.

6.3.6. Componentes propios del proyecto

Además de las librerías externas y propias de ESP-IDF, el firmware cuenta con componentes desarrollados específicamente para este proyecto. Estos módulos permiten separar las funciones principales del sistema, facilitando la organización, depuración y mantenimiento del código.

Componente	Archivo	Función en el proyecto
Reloj RTC	<code>reloj.h</code> , <code>reloj.c</code>	Inicializa el DS3231M, permite leer la fecha y hora actual, y convierte dicha información a formato <i>timestamp</i> UNIX para asociar cada medición con un tiempo determinado.
SD Card	<code>sdcard.h</code> , <code>sdcard.c</code>	Monta la tarjeta SD, crea archivos como <code>dataalog.csv</code> y <code>temp.csv</code> , guarda los datos medidos y maneja una cola de datos pendientes para su posterior envío.
Sensor RS485	<code>sensor_rs485.h</code> , <code>sensor_rs485.c</code>	Lee el sensor de nivel ultrasónico/radar mediante comunicación RS485. El módulo se encarga de enviar la solicitud de lectura y procesar la respuesta recibida desde el sensor.
Módem DTU	<code>modem_dtu.h</code> , <code>modem_dtu.c</code>	Envía la trama de datos al servidor en el formato <code>d=timestamp,nivel</code> y lee la respuesta recibida desde el módem DTU para verificar si el envío fue exitoso.
LED Blink	<code>led_blink.h</code> , <code>led_blink.c</code>	Hace parpadear el LED conectado al GPIO21 como indicador visual del funcionamiento del sistema. Se utiliza, por ejemplo, para indicar que la lectura del sensor fue realizada correctamente.

Cuadro 10: Componentes propios desarrollados para el proyecto.

6.4. Módulos desarrollados

El firmware del proyecto fue desarrollado de forma modular, separando las funciones principales en archivos independientes. Esta organización permite que el archivo `main.c` contenga principalmente el flujo general del programa, mientras que cada componente se encarga de una función específica del sistema.

Los módulos desarrollados se organizaron de la siguiente manera:

```
src/  
  main.c  
  
components/  
  reloj/  
    reloj.h  
    reloj.c  
  
  sdcard/  
    sdcard.h  
    sdcard.c  
  
  sensor_rs485/  
    sensor_rs485.h  
    sensor_rs485.c  
  
  modem_dtu/  
    modem_dtu.h  
    modem_dtu.c  
  
  led/  
    led_blink.h  
    led_blink.c
```

A continuación, se describe la función principal de cada módulo desarrollado:

Módulo	Función principal
<code>main.c</code>	Contiene el flujo principal del programa. Se encarga de inicializar los módulos del sistema, coordinar la lectura del sensor, guardar los datos en la tarjeta SD, enviar la información mediante el módem DTU y controlar la ejecución periódica del sistema.
<code>reloj</code>	Módulo encargado del manejo del DS3231M. Permite inicializar el reloj, leer la fecha y hora actual, y convertir dicha información a un formato de tiempo útil para registrar cada medición.
<code>sdcard</code>	Módulo encargado del manejo de la tarjeta SD. Permite montar la memoria, crear archivos de registro, guardar mediciones y manejar datos pendientes de envío en caso de falla de comunicación.
<code>sensor_rs485</code>	Módulo encargado de la comunicación con el sensor de nivel mediante RS485. Realiza la solicitud de lectura, recibe la respuesta del sensor y obtiene el valor de nivel medido.
<code>modem_dtu</code>	Módulo encargado de la comunicación con el módem DTU. Permite enviar al servidor la trama de datos generada por el sistema y leer la respuesta recibida para verificar si el envío fue exitoso.
<code>led</code>	Módulo encargado del indicador visual del sistema. Permite hacer parpadear un LED para señalar estados de funcionamiento, por ejemplo, cuando una lectura del sensor fue realizada correctamente.

Cuadro 11: Módulos desarrollados en el firmware del proyecto.

7. Flujo conceptual del algoritmo final

El algoritmo final del sistema busca realizar la adquisición de datos de forma ordenada, procesar la información obtenida y ejecutar las acciones correspondientes, como almacenar los datos, transmitirlos o activar alguna salida.

7.1. Descripción del flujo

A continuación, se presenta el diagrama de flujo que ilustra la secuencia lógica de ejecución del firmware en el ESP32:

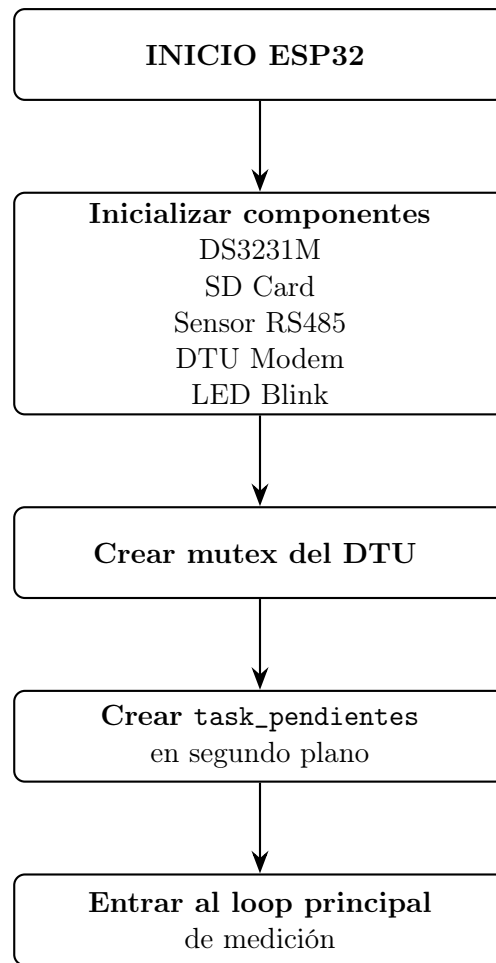


Figura 8: Secuencia inicial de ejecución del firmware en el ESP32.

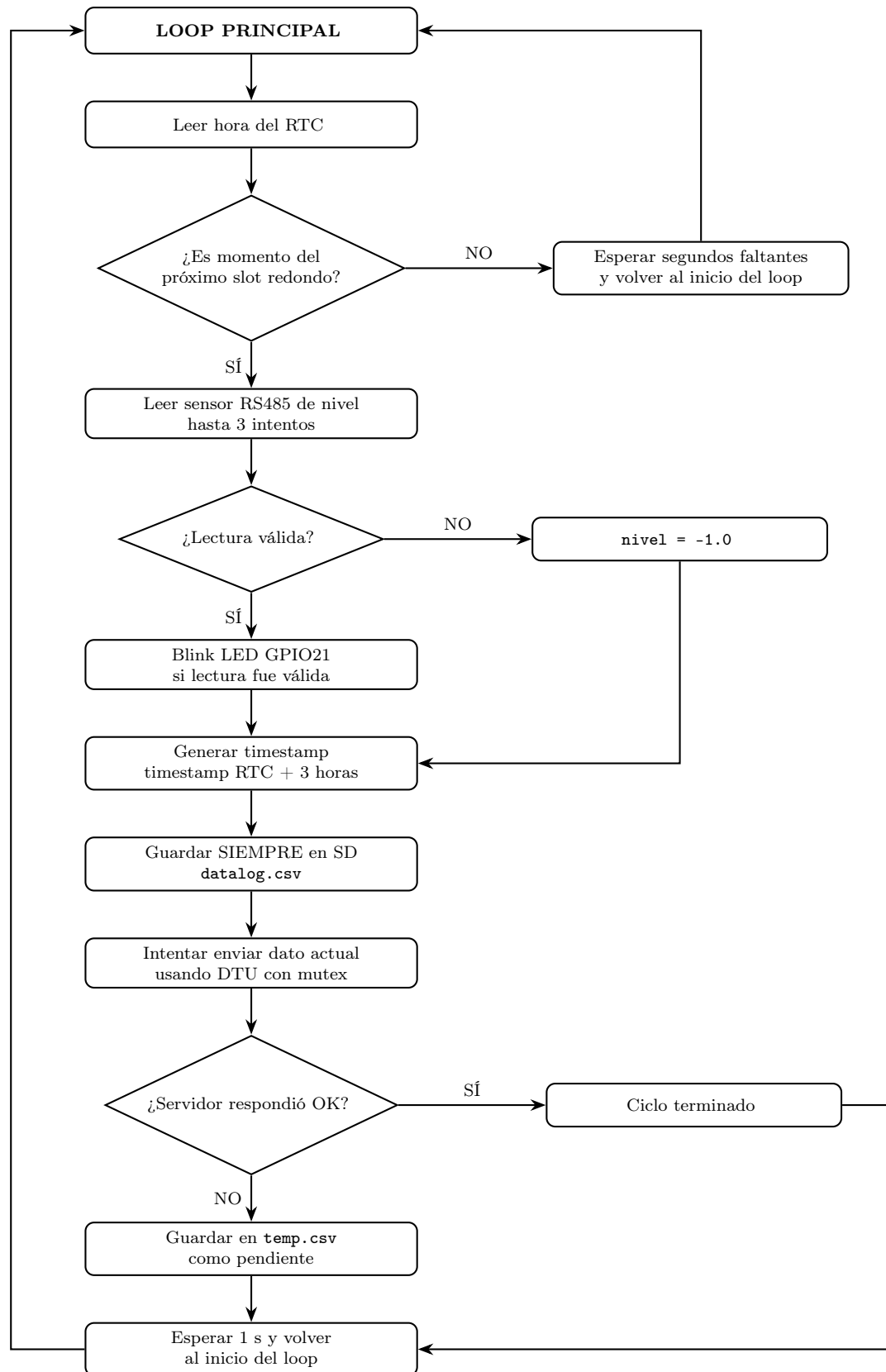


Figura 9: Secuencia lógica del loop principal de medición, almacenamiento y envío de datos.

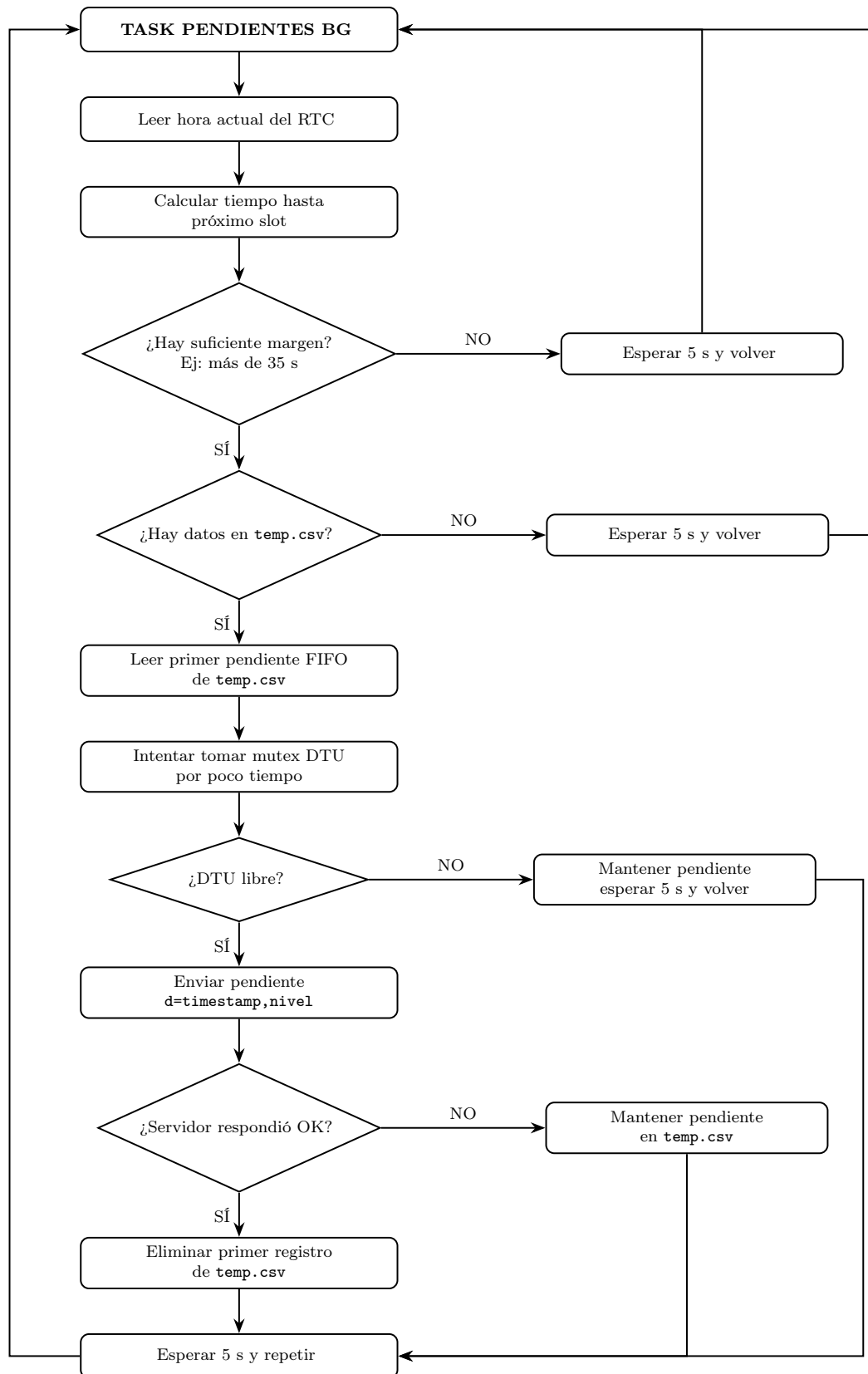


Figura 10: Flujo de la tarea en segundo plano para el reenvío de datos pendientes.

8. Resultados de pruebas aisladas de módulos del sistema

Antes de realizar la integración completa del firmware, se ejecutaron pruebas individuales sobre cada módulo del sistema. Estas pruebas permitieron verificar el funcionamiento independiente del sensor de nivel, el reloj de tiempo real, la tarjeta SD y el módem DTU [5]. De esta manera, se redujo la posibilidad de errores durante la integración final del sistema.

8.1. Prueba aislada del sensor de nivel RS485

El objetivo de esta prueba fue verificar que el sensor de nivel entregue una medición válida mediante comunicación RS485. Para ello, se utilizó una trama Modbus RTU de lectura y se observó la respuesta recibida por el ESP32-S3 [1].

Cuadro 12: Resultados de prueba aislada del sensor de nivel RS485.

Prueba	Condición aplicada	Resultado obtenido	Estado
Prueba 1	Sensor conectado al bus RS485	Se recibió una trama válida del sensor y se obtuvo el nivel en metros.	Correcto
Prueba 2	Lectura repetida cada cierto intervalo	El valor medido se mantuvo coherente y estable.	Correcto
Prueba 3	Sensor desconectado	El sistema detectó ausencia de respuesta y mostró mensaje de error.	Correcto
Prueba 4	Reintento de lectura	Cuando la primera lectura falló, el sistema realizó nuevos intentos hasta obtener una lectura válida.	Correcto

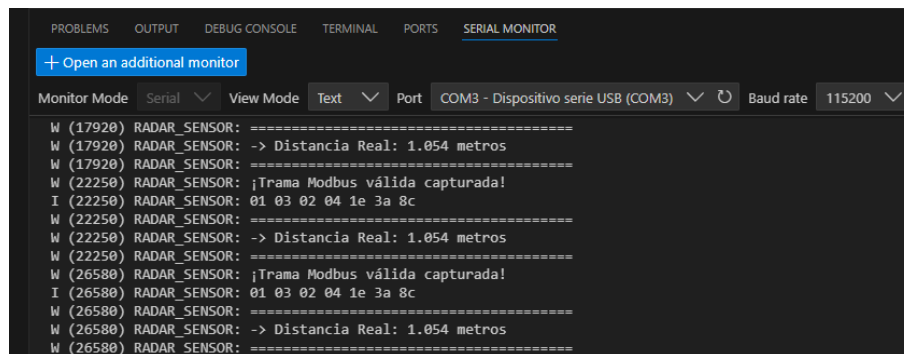


Figura 11: Lectura del sensor de nivel RS485 observada en el monitor serial. Se verifica la recepción de una trama válida y la conversión del valor medido a metros.

8.2. Prueba aislada del DS3231M

El objetivo de esta prueba fue verificar que el módulo DS3231M permita leer la fecha y hora actual mediante comunicación I2C.

Cuadro 13: Resultados de prueba aislada del RTC DS3231.

Prueba	Condición aplicada	Resultado obtenido	Estado
Prueba 1	RTC conectado al bus I2C	El ESP32-S3 detectó el módulo y pudo leer la hora actual.	Correcto
Prueba 2	Lectura periódica de hora	La hora avanzó correctamente segundo a segundo.	Correcto
Prueba 3	Conversión a timestamp	La fecha y hora del RTC fueron convertidas a timestamp UNIX.	Correcto
Prueba 4	Cálculo del próximo intervalo	El firmware calculó correctamente el tiempo restante hasta el próximo slot de medición.	Correcto

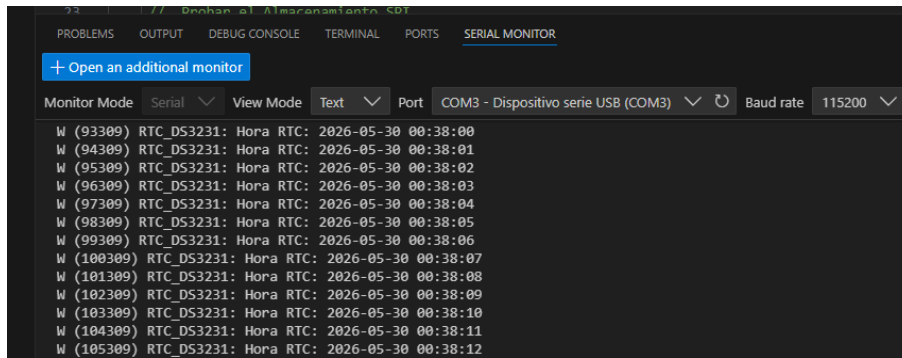


Figura 12: Prueba aislada del DS3231M. La imagen muestra la lectura de fecha y hora utilizada para generar el timestamp de cada medición.

8.3. Prueba aislada de la tarjeta SD

El objetivo de esta prueba fue verificar la detección, montaje, escritura y lectura de archivos en la tarjeta microSD. La tarjeta SD se utiliza para almacenar el historial de mediciones y la cola de datos pendientes.

Cuadro 14: Resultados de prueba aislada de la tarjeta SD.

Prueba	Condición aplicada	Resultado obtenido	Estado
Prueba 1	Tarjeta SD insertada	El pin de detección indicó presencia de tarjeta.	Correcto
Prueba 2	Montaje de la SD por SPI	La tarjeta fue montada correctamente en <code>/sdcard</code> .	Correcto
Prueba 3	Escritura de archivo	Se creó un archivo de prueba y se escribieron datos correctamente.	Correcto
Prueba 4	Lectura de archivo	El contenido escrito fue leído correctamente desde la SD.	Correcto

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - Dispositivo serie USB (COM3)
---- Opened the serial port COM3 ----
I (6566) MAIN: Hay SD, puedo montar y guardar datos
I (6566) SDCARD: =====
I (6566) SDCARD: INICIANDO PRUEBA SD
I (6566) SDCARD: =====
E (6566) spi: spi_bus_initialize(816): SPI bus already initialized.
I (6576) SDCARD: SPI OK
I (6576) SDCARD: Montando SD...
I (6576) sdsd_transaction: cmd=52, R1 response: command not supported
I (6626) sdsd_transaction: cmd=5, R1 response: command not supported
I (6656) SDCARD: SD montada correctamente
Name: SD
Type: SDHC
Speed: 10.00 MHz (limit: 10.00 MHz)
Size: 14910MB
CSD: ver=2, sector_size=512, capacity=30535680 read_bl_len=9
SSR: bus_width=1
I (6656) SDCARD: Creando archivo...
I (6686) SDCARD: Archivo escrito
I (6686) SDCARD: Leyendo archivo...

===== CONTENIDO =====
Hola desde ESP32-S3
Prueba SD correcta
===== FIN =====

I (6686) SDCARD: Lectura OK
I (6696) SDCARD: SD desmontada
I (6696) SDCARD: PRUEBA FINALIZADA OK
---- Closed the serial port COM3 ----

```

Figura 13: Prueba aislada de la tarjeta microSD. Se verifica el montaje correcto de la tarjeta y el guardado de datos, lectura y escritura.

8.4. Prueba aislada del módem DTU

El objetivo de esta prueba fue comprobar la comunicación entre el ESP32-S3 [1] y el módem DTU mediante RS485. El módem se encarga de enviar los datos al servidor utilizando la trama definida para el sistema.

Cuadro 15: Resultados de prueba aislada del módem DTU.

Prueba	Condición aplicada	Resultado obtenido	Estado
Prueba 1	Inicialización UART RS485	El puerto UART del ESP32-S3 fue configurado correctamente.	Correcto
Prueba 2	Envío de trama válida	Se envió la trama <code>d=timestamp,nivel</code> al servidor.	Correcto
Prueba 3	Respuesta del servidor	El módem recibió respuesta OK luego del envío correcto.	Correcto
Prueba 4	Módem desconectado	El sistema detectó ausencia de respuesta y conservó el dato como pendiente.	Correcto

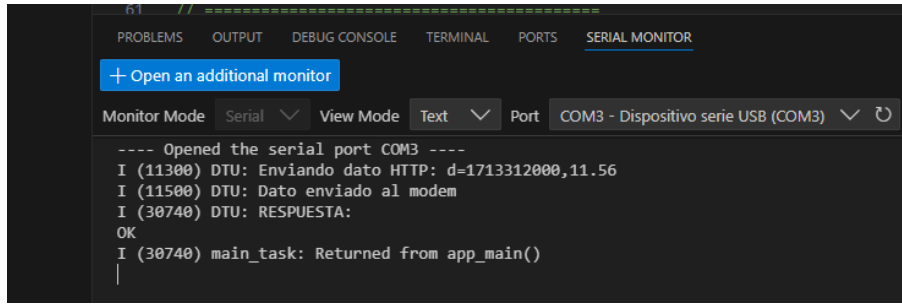


Figura 14: Prueba aislada del módem DTU. Se observa el envío de la trama `d=timestamp,nivel` y la respuesta `OK` recibida desde el servidor.

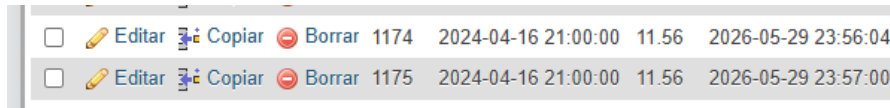


Figura 15: Prueba aislada del módem DTU. Se observa el envío de la trama `d=timestamp,nivel` ahora visto desde el servidor.

8.5. Prueba integrada del sistema completo

En esta prueba se verificó el funcionamiento conjunto de todos los módulos del sistema, desde la lectura del sensor de nivel hasta el almacenamiento local y el envío de datos al servidor.

Prueba	Condición aplicada	Resultado obtenido	Estado
Prueba 1	Sistema encendido con todos los módulos conectados	El ESP32-S3 inicializó correctamente el RTC, SD, sensor RS485, módem DTU y LED indicador.	Correcto
Prueba 2	Lectura periódica del sensor de nivel	El sistema obtuvo lecturas válidas del nivel en intervalos definidos.	Correcto
Prueba 3	Registro local en tarjeta microSD	Cada medición fue almacenada en el archivo <code>datalog.csv</code> .	Correcto
Prueba 4	Envío de datos al servidor	El dato fue transmitido con la trama <code>d=timestamp,nivel</code> y recibido correctamente por el servidor.	Correcto
Prueba 5	Desconexión temporal del módem o falla de comunicación	El sistema almacenó el dato en <code>temp.csv</code> y lo reenvió luego del restablecimiento de la comunicación.	Correcto

Cuadro 16: Resultados de la prueba integrada del sistema completo.

9. Estado actual del proyecto

El proyecto se encuentra actualmente finalizado y en funcionamiento correcto. Se logró desarrollar e integrar una estación de medición de nivel basada en un microcontrolador ESP32-S3 [1], utilizando el framework ESP-IDF [2] dentro del entorno PlatformIO [3]. El sistema permite realizar mediciones periódicas del nivel mediante un sensor de comunicación RS485 [6], registrar los datos cada 2 minutos en una tarjeta microSD y transmitirlos hacia un servidor remoto mediante un módem DTU [5].

Durante la etapa final del desarrollo se verificó el funcionamiento individual de cada módulo del sistema: sensor de nivel RS485, reloj de tiempo real DS3231M, tarjeta SD y módem DTU. Posteriormente, estos módulos fueron integrados en un firmware principal, logrando una operación coordinada y estable.

El sistema realiza la lectura del sensor de nivel en intervalos definidos mediante el reloj DS3231M [7], generando un registro compuesto por el timestamp y el valor de nivel medido. Cada medición se almacena localmente en el archivo `data.log.csv`, asegurando que exista un respaldo físico de los datos en la tarjeta SD. Además, el dato es enviado al servidor con el formato:

```
d=timestamp,nivel
```

Una de las mejoras más importantes implementadas fue el mecanismo de almacenamiento y reenvío de datos pendientes, conocido como *Store and Forward*. Este mecanismo permite que, en caso de falla de comunicación con el módem o ausencia de conexión con el servidor, los datos no enviados sean almacenados temporalmente en el archivo `temp.csv`. Cuando la comunicación se restablece, el sistema reenvía los datos pendientes de forma automática y conserva el orden de generación de las mediciones.

También se incorporó una tarea en segundo plano para el reenvío de datos pendientes, evitando que esta operación interfiera directamente con el ciclo principal de medición. Para evitar conflictos en el uso del módem DTU [5], se implementó un mecanismo de sincronización mediante un mutex, de modo que el ciclo principal y la tarea de pendientes no utilicen el módem al mismo tiempo.

Como indicador visual, se agregó un módulo LED conectado al GPIO21, el cual parpadea cuando el sensor entrega una lectura válida. Esto permite verificar rápidamente que el sistema está realizando mediciones correctamente sin necesidad de observar constantemente el monitor serial.

En las pruebas finales se comprobó que el sistema mide correctamente el nivel, guarda los registros en la tarjeta SD, envía los datos al servidor y conserva los datos no enviados cuando se simula una interrupción de comunicación. Una vez restablecida la conexión, los datos pendientes son reenviados correctamente al servidor, manteniendo el timestamp original de la medición.

Por lo tanto, se concluye que el sistema desarrollado cumple con los objetivos planteados, ya que permite la medición, almacenamiento local, transmisión remota y recuperación de datos ante fallas de comunicación, funcionando de manera estable en las pruebas realizadas.

10. Repositorio GitHub

El código fuente y los archivos del proyecto fueron organizados en un repositorio de GitHub, con el objetivo de mantener ordenada la documentación técnica, el firmware desarrollado, los archivos de hardware y las evidencias de prueba del sistema.

El repositorio se encuentra estructurado en carpetas principales, separando el firmware final, los códigos utilizados para pruebas aisladas, los archivos de diseño electrónico y las evidencias audiovisuales del funcionamiento del sistema.

- **Firmware_Final**: contiene el firmware principal del sistema de adquisición de nivel, desarrollado en PlatformIO con framework ESP-IDF.
- **Firmware_Pruebas**: contiene los códigos utilizados para realizar pruebas aisladas de los módulos, tales como sensor RS485, RTC DS3231, tarjeta microSD, módem DTU y LED indicador.
- **Hardware_Kicad**: contiene los archivos correspondientes al diseño electrónico del proyecto, incluyendo esquemáticos, PCB, archivos de fabricación y documentación asociada al hardware.
- **Evidencias_Pruebas**: contiene capturas y archivos `README.md` con enlaces a videos de prueba del sistema. Estos videos documentan el funcionamiento del LED indicador, la tarjeta microSD, el RTC DS3231, el sistema integrado y el mecanismo *Store and Forward*.

Repositorio del proyecto:

https://github.com/HecVelaz/Proyecto-Mburica-/tree/main/02_Estacion/Proyecto3__Sistema_de_adquisicion_de_Nivel

La inclusión de la carpeta **Evidencias_Pruebas** permite complementar el informe técnico con material audiovisual. Los videos no fueron cargados directamente al repositorio para evitar aumentar su tamaño, por lo que se encuentran alojados externamente y enlazados mediante archivos `README.md`. Estas evidencias permiten verificar visualmente las pruebas realizadas durante el desarrollo e integración del sistema.

11. Conclusión

El desarrollo del proyecto permitió implementar una estación de adquisición de datos de nivel basada en el microcontrolador ESP32-S3, utilizando el framework ESP-IDF en el entorno PlatformIO. El sistema fue diseñado para medir el nivel mediante un sensor con comunicación RS485, registrar localmente las mediciones en una tarjeta microSD y transmitir los datos hacia un servidor remoto mediante un módem DTU.

Durante el desarrollo se trabajó de manera modular, separando el firmware en distintos componentes: reloj RTC, tarjeta SD, sensor RS485, módem DTU, LED indicador y programa principal. Esta estructura permitió organizar mejor el código, facilitar las pruebas individuales y simplificar la integración final del sistema.

Se logró integrar correctamente el reloj de tiempo real DS3231 para generar marcas de tiempo confiables en cada medición. A partir de esta información, el sistema registra cada dato con su correspondiente timestamp, permitiendo identificar el momento real en que fue tomada la lectura. Además, se implementó una lógica de medición periódica sincronizada con intervalos definidos, lo que permite realizar adquisiciones de manera ordenada y repetitiva.

El sensor de nivel RS485 fue configurado y probado mediante comunicación UART, obteniéndose lecturas válidas del nivel medido. Estas lecturas son almacenadas de forma permanente en la tarjeta SD dentro del archivo `datalog.csv`, asegurando que exista un respaldo local de toda la información registrada por la estación.

También se implementó la comunicación con el módem DTU para el envío de los datos al servidor. La trama utilizada para la transmisión fue definida con el formato:

```
d=timestamp,nivel
```

Una de las partes más importantes del proyecto fue la incorporación del sistema de almacenamiento y reenvío de datos pendientes, conocido como *Store and Forward*. Este mecanismo permite que, si ocurre una falla de comunicación o el servidor no responde, el dato no se pierda, sino que sea almacenado temporalmente en el archivo `temp.csv`. Cuando la comunicación se restablece, los datos pendientes son reenviados automáticamente al servidor, conservando el orden en que fueron generados.

Para evitar conflictos en el uso del módem DTU, se implementó una tarea en segundo plano encargada del reenvío de datos pendientes, junto con un mecanismo de sincronización mediante mutex. De esta manera, el ciclo principal de medición y la tarea de reenvío no utilizan el módem al mismo tiempo. Además, se agregó un LED indicador conectado al GPIO21, el cual permite verificar visualmente cuándo el sensor entrega una lectura válida.

Las pruebas realizadas demostraron que los módulos principales funcionan correctamente de forma aislada e integrada. Se verificó la lectura del sensor, el funcionamiento del RTC, el montaje y escritura en la tarjeta SD, el envío de datos al servidor, el almacenamiento de datos pendientes y su posterior reenvío. También se comprobó que el servidor recibe los datos correctamente y que el campo `timestamp` conserva el tiempo real de medición.

En conclusión, el proyecto cumplió con los objetivos planteados, ya que se obtuvo un sistema funcional capaz de medir, almacenar y transmitir datos de nivel de manera automática. Además, la implementación del respaldo local y del sistema *Store and Forward* mejora la confiabilidad del sistema frente a fallas de comunicación, evitando la pérdida de información. Por lo tanto, la estación desarrollada representa una base sólida para aplicaciones de monitoreo remoto de nivel y puede ser ampliada en el futuro con mejoras como alimentación autónoma, carcasa de protección, monitoreo web en tiempo real y validación prolongada en campo.

12. Trabajos futuros

Como trabajos futuros se propone implementar una carcasa de protección para operación en campo, optimizar el sistema de alimentación mediante panel solar o batería de respaldo, mejorar la interfaz de monitoreo y realizar pruebas prolongadas

en condiciones reales de instalación. Además, se recomienda actualizar la estructura de la base de datos para que los campos representen directamente las variables medidas por el sistema final.

También se propone el desarrollo de una plataforma web propia para visualizar los datos almacenados en el servidor. Esta interfaz permitiría consultar las mediciones de nivel en tiempo real, revisar datos históricos, mostrar gráficos de evolución temporal y facilitar el acceso a la información generada por la estación.

Actualmente, como referencia, Grafana representa una alternativa adecuada para este tipo de visualización, ya que permite mostrar los datos disponibles en el servidor mediante gráficos temporales, indicadores de estado y mapas de ubicación. La Figura 16 muestra un ejemplo de lo que podría visualizarse utilizando los datos almacenados en el servidor; sin embargo, este panel no forma parte del desarrollo principal del presente proyecto, sino que se incluye como referencia para futuras mejoras del sistema.

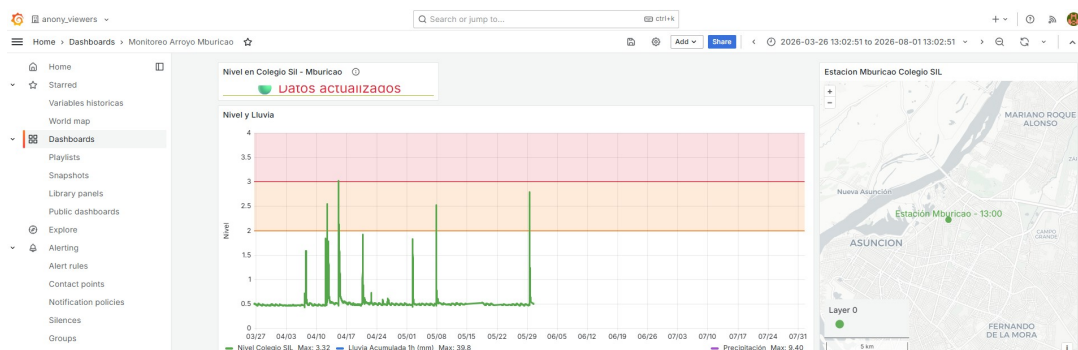


Figura 16: Ejemplo de visualización posible mediante Grafana utilizando los datos almacenados en el servidor. La imagen muestra cómo podrían representarse las mediciones de nivel mediante una gráfica temporal, un indicador de actualización y un mapa de ubicación. Este panel se presenta únicamente como referencia para trabajos futuros y no forma parte del desarrollo principal del proyecto.

13. Referencias

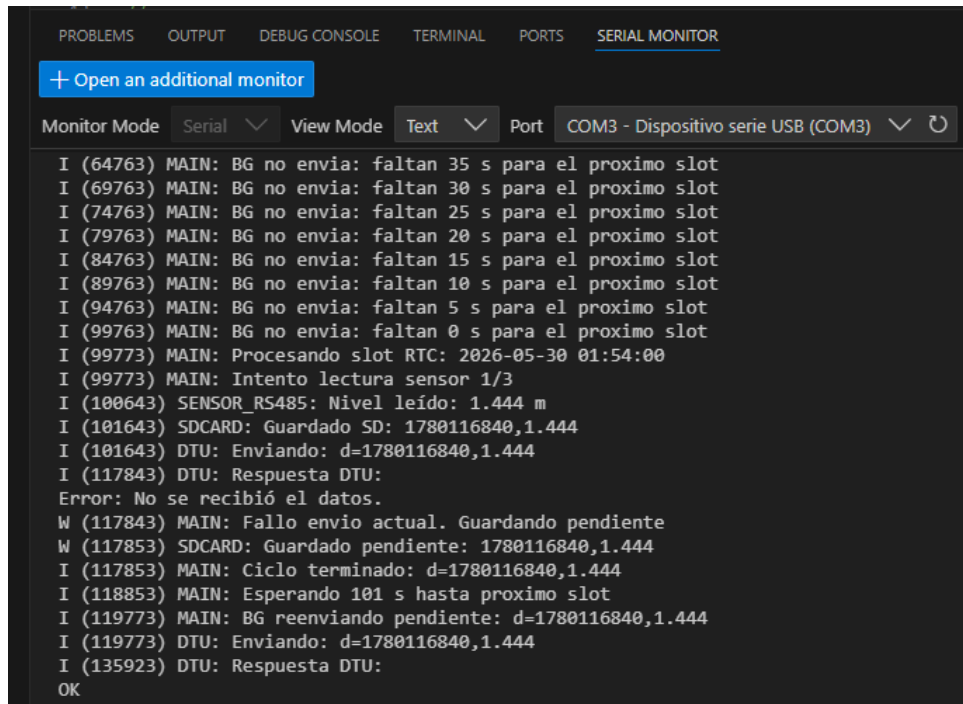
- [1] Espressif Systems, *ESP32-S3-WROOM-1 and ESP32-S3-WROOM-1U Datasheet*, versión 1.8. Disponible en: https://www.mouser.es/datasheet/3/1574/1/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- [2] Espressif Systems, *ESP-IDF Programming Guide: Set up the tools in Visual Studio Code*. Disponible en: <https://docs.espressif.com/projects/esp-idf/en/v4.2.1/esp32/get-started/vscode-setup.html>
- [3] PlatformIO, *PlatformIO IDE for Visual Studio Code*. Disponible en: <https://docs.platformio.org/en/latest/integration/ide/vscode.html>
- [4] M. Aguilar et al., *Machine Learning Models for Water Level Prediction in Rapid Urban Streams: Case of Mburicao, Asunción, Paraguay*, IEEE Xplore, 2025. Disponible en: <https://ieeexplore.ieee.org/document/11420547>

- [5] USR IOT, *GSM GPRS Modem, Serial RS232/RS485 to GSM DTU Modem*. Disponible en: <https://en.usr.cn/GSM-Modem-Serial-RS232-RS485-GPRS.html>
- [6] AliExpress, *Sensor de nivel ultrasónico con comunicación RS485*. Ficha comercial del producto. Disponible en: <https://es.aliexpress.com/item/1005008225897752.html?gatewayAdapt=glo2esp> Consultado el 30 de mayo de 2026.
- [7] Analog Devices, *DS3231M Low-Cost, Extremely Accurate, I2C Real-Time Clock Datasheet*. Disponible en: <https://www.analog.com/media/en/technical-documentation/data-sheets/DS3231M.pdf>
- [8] Molex, *MicroSD Memory Card Connector, Part Number 1040310811 Datasheet*. Disponible en: <https://www.molex.com/en-us/products/part-detail/1040310811?display=pdf>
- [9] GCT, *USB4930-00-A USB Type-C 2.0 Connector Datasheet*. Disponible en: <https://www.mouser.sk/datasheet/3/1507/1/usb4930.pdf>
- [10] Shandong Puhui Electric Technology Co., Ltd., *INFRARED IC HEATER User Manual (Model: T962/T962A/T962C)*. Puhui Electronic Technology. Disponible en: <https://en.tech168.cn/php/download-detail.php?id=5> Consultado el 30 de mayo de 2026.
- [11] Marsriva, *Mini UPS Power Supply System*, disponible en: https://www.marsriva.com/Products_item_2/157.html

14. Anexos

14.1. Capturas adicionales del sistema

En esta sección se presentan capturas complementarias obtenidas durante el desarrollo, prueba e integración del sistema. Estas imágenes permiten evidenciar el funcionamiento de los distintos módulos implementados, tales como la lectura del sensor, el almacenamiento en tarjeta SD, el envío de datos al servidor y la recuperación de datos pendientes.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - Dispositivo serie USB (COM3)
I (64763) MAIN: BG no envia: faltan 35 s para el proximo slot
I (69763) MAIN: BG no envia: faltan 30 s para el proximo slot
I (74763) MAIN: BG no envia: faltan 25 s para el proximo slot
I (79763) MAIN: BG no envia: faltan 20 s para el proximo slot
I (84763) MAIN: BG no envia: faltan 15 s para el proximo slot
I (89763) MAIN: BG no envia: faltan 10 s para el proximo slot
I (94763) MAIN: BG no envia: faltan 5 s para el proximo slot
I (99763) MAIN: BG no envia: faltan 0 s para el proximo slot
I (99773) MAIN: Procesando slot RTC: 2026-05-30 01:54:00
I (99773) MAIN: Intento lectura sensor 1/3
I (100643) SENSOR_RS485: Nivel leído: 1.444 m
I (101643) SDCARD: Guardado SD: 1780116840,1.444
I (101643) DTU: Enviando: d=1780116840,1.444
I (117843) DTU: Respuesta DTU:
Error: No se recibió el datos.
W (117843) MAIN: Fallo envio actual. Guardando pendiente
W (117853) SDCARD: Guardado pendiente: 1780116840,1.444
I (117853) MAIN: Ciclo terminado: d=1780116840,1.444
I (118853) MAIN: Esperando 101 s hasta proximo slot
I (119773) MAIN: BG reenviando pendiente: d=1780116840,1.444
I (119773) DTU: Enviando: d=1780116840,1.444
I (135923) DTU: Respuesta DTU:
OK
```

Figura 17: Evidencia del funcionamiento del mecanismo *Store and Forward* en el monitor serial. Se observa que el sistema realiza la lectura del sensor de nivel, guarda el dato en la tarjeta SD y luego intenta enviarlo mediante el módem DTU. Al recibir una respuesta de error del servidor, el dato es almacenado como pendiente en `temp.csv`. Posteriormente, la tarea en segundo plano reenvía el dato pendiente y recibe la respuesta `OK`, confirmando su transmisión correcta.



Figura 18: Montaje de prueba del sistema de adquisición de nivel. Se observa el sensor de nivel conectado a la placa principal basada en ESP32-S3 y al módem DTU GPRS mediante comunicación RS485. El montaje permitió verificar la lectura del sensor, el procesamiento del dato en la placa y el envío de la información hacia el servidor remoto.

Servidor: localhost:3306 » Base de datos: NivelesMburicao » Tabla: SilEvelyn

Examinar Estructura SQL Buscar Insertar Exportar Importar Operaciones Disparadores

Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

				id	timestamp	lluvia	created_at
☐	Editar	Copiar	Borrar	1184	2026-05-30 02:10:00	0.302	2026-05-30 01:10:14
☐	Editar	Copiar	Borrar	1183	2026-05-30 02:08:00	0.303	2026-05-30 01:08:14
☐	Editar	Copiar	Borrar	1182	2026-05-30 02:06:00	2.062	2026-05-30 01:06:14
☐	Editar	Copiar	Borrar	1181	2026-05-30 02:04:00	0.908	2026-05-30 01:04:14
☐	Editar	Copiar	Borrar	1180	2026-05-30 02:02:00	1.508	2026-05-30 01:02:13
☐	Editar	Copiar	Borrar	1179	2026-05-30 02:00:00	1.365	2026-05-30 01:00:13
☐	Editar	Copiar	Borrar	1178	2026-05-30 01:58:00	1.318	2026-05-30 00:58:14
☐	Editar	Copiar	Borrar	1177	2026-05-30 01:56:00	1.307	2026-05-30 00:56:14
☐	Editar	Copiar	Borrar	1176	2026-05-30 01:54:00	1.444	2026-05-30 00:54:32
☐	Editar	Copiar	Borrar	1173	2026-05-30 00:28:00	1.056	2026-05-29 23:28:14
☐	Editar	Copiar	Borrar	1172	2026-05-30 00:24:00	1.055	2026-05-29 23:24:16
☐	Editar	Copiar	Borrar	1171	2026-05-30 00:22:00	1.054	2026-05-29 23:22:13
☐	Editar	Copiar	Borrar	1170	2026-05-30 00:20:00	1.055	2026-05-29 23:20:14
☐	Editar	Copiar	Borrar	1169	2026-05-30 00:18:00	1.054	2026-05-29 23:18:14
☐	Editar	Copiar	Borrar	1168	2026-05-30 00:16:00	1.055	2026-05-29 23:16:14
☐	Editar	Copiar	Borrar	1167	2026-05-30 00:14:00	1.055	2026-05-29 23:14:14
☐	Editar	Copiar	Borrar	1166	2026-05-30 00:12:00	1.055	2026-05-29 23:12:13
☐	Editar	Copiar	Borrar	1165	2026-05-30 00:10:00	1.054	2026-05-29 23:10:14
☐	Editar	Copiar	Borrar	1164	2026-05-30 00:08:00	1.055	2026-05-29 23:08:13
☐	Editar	Copiar	Borrar	1163	2026-05-30 00:06:00	1.054	2026-05-29 23:06:14
☐	Editar	Copiar	Borrar	1162	2026-05-30 00:04:00	1.055	2026-05-29 23:04:14
☐	Editar	Copiar	Borrar	1161	2026-05-30 00:02:00	1.055	2026-05-29 23:02:14
☐	Editar	Copiar	Borrar	1160	2026-05-30 00:00:00	1.054	2026-05-29 23:00:14
☐	Editar	Copiar	Borrar	1159	2026-05-29 23:58:00	1.055	2026-05-29 22:58:14
☐	Editar	Copiar	Borrar	1158	2026-05-29 23:56:00	1.054	2026-05-29 22:56:14

Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar

Figura 19: Registros almacenados en la base de datos del servidor luego del envío realizado por la estación de adquisición de nivel. Se observan los campos `timestamp`, correspondiente al instante de medición, `lluvia`, columna reutilizada en esta etapa para almacenar el valor de nivel medido, y `created_at`, que indica el momento en que el registro fue insertado en la base de datos con un desfase aproximado de una hora.

☐	Editar	Copiar	Borrar	1188	2026-05-30 02:22:00	0.301	2026-05-30 01:22:14
☐	Editar	Copiar	Borrar	1187	2026-05-30 02:16:00	0.301	2026-05-30 01:21:30
☐	Editar	Copiar	Borrar	1186	2026-05-30 02:14:00	0.301	2026-05-30 01:14:13
☐	Editar	Copiar	Borrar	1185	2026-05-30 02:12:00	0.302	2026-05-30 01:12:13

Figura 20: Estado inicial de la base de datos durante la prueba Store and Forward. Se observa una interrupción temporal en la secuencia de registros entre 02:16 y 02:22, correspondiente a datos pendientes de reenvío.

☐		Editar		Copiar		Borrar	1188	2026-05-30 02:22:00	0.301	2026-05-30 01:22:14
☐		Editar		Copiar		Borrar	1189	2026-05-30 02:18:00	0.301	2026-05-30 01:22:34
☐		Editar		Copiar		Borrar	1187	2026-05-30 02:16:00	0.301	2026-05-30 01:21:30
☐		Editar		Copiar		Borrar	1186	2026-05-30 02:14:00	0.301	2026-05-30 01:14:13

Figura 21: Reenvío progresivo de datos pendientes provenientes de la cola almacenada en la tarjeta SD. Se observa la inserción posterior del registro correspondiente a las 02:18 en la base de datos del servidor, conservando su timestamp original.

☐		Editar		Copiar		Borrar	1188	2026-05-30 02:22:00	0.301	2026-05-30 01:22:14
☐		Editar		Copiar		Borrar	1190	2026-05-30 02:20:00	0.301	2026-05-30 01:22:55
☐		Editar		Copiar		Borrar	1189	2026-05-30 02:18:00	0.301	2026-05-30 01:22:34
☐		Editar		Copiar		Borrar	1187	2026-05-30 02:16:00	0.301	2026-05-30 01:21:30
☐		Editar		Copiar		Borrar	1186	2026-05-30 02:14:00	0.301	2026-05-30 01:14:13

Figura 22: Evidencia del reenvío de datos pendientes mediante el mecanismo *Store and Forward*. Se añade 02:20 y finaliza la subida de los datos pendientes.

En la Figura 23 se muestra una captura del archivo `datalog.csv` generado en la tarjeta microSD. Este archivo almacena el historial de mediciones realizadas por la estación, utilizando el formato `timestamp,nivel`.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1		Timestamp,nivel														
2		1779589786,0.257														
3		1779590107,0.257														
4		1779590429,0.257														
5		1779590750,0.257														
6		1779591072,0.257														
7		1779591395,0.257														
8		1779591717,0.257														
9		1779592039,0.257														
10		1779592361,0.260														
11		1779592684,0.260														
12		1779617400,0.932														
13		1779617700,0.931														
14		1779617760,0.932														
15		1779617880,0.933														
16		1779617940,0.932														
17		1779618060,0.931														
18		1779618120,0.932														
19		1779618180,0.932														
20		1779618240,0.932														
21		1779618300,0.932														

Figura 23: Captura del archivo `datalog.csv` generado en la tarjeta microSD. Se observa el registro local de las mediciones de nivel, donde cada fila contiene el timestamp de adquisición y el valor de nivel medido en metros.